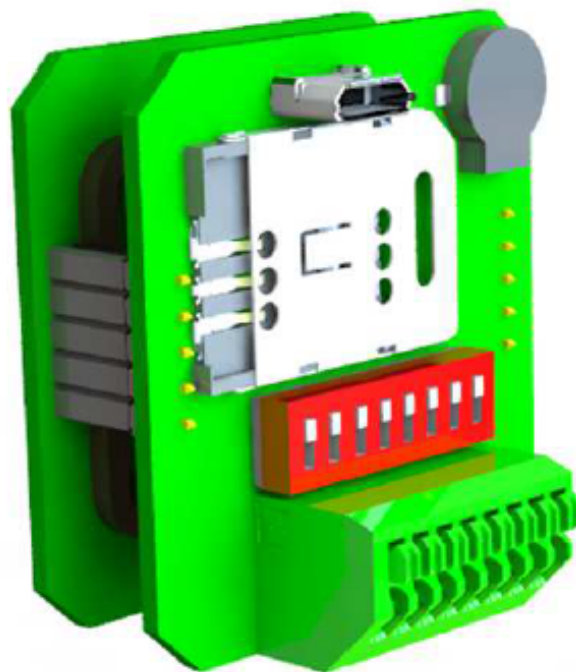


TWN4 Palon OSDP OSS Extension

Protocol Definition

DocRev8, April 1, 2025



ELATEC GmbH

Revision History:

DocRev	Description and Changes	Date
1	First draft. Describes basic principles of file access instructions embedded in OSDP message. OSDP standard v2.1.7 is used at the time of writing this guide.	16.08.2018
2	Pause transponder search for some seconds to allow Host to process transponder. Added command OSS_CreateFile. Changed Offset endianness for OSS_ReadData and OSS_WriteData commands for more optimized processing.	13.09.2018
3	New command OSS_Commit. Commands OSS_CreateFile and OSS_DeleteFile are no longer supported. The <i>FileNo</i> is now parameter of the command. Parameter <i>Length</i> is extended to a 16-bit value	21.01.2019

Contents

1	Overview	4
2	Message Structure	5
3	OSS Commands	6
3.1	OSS_GetFileSize	6
3.2	OSS_ReadData	7
3.3	OSS_WriteData	7
3.4	OSS_Commit	8
4	Disclaimer	9

1 Overview

OSS-SO is an offline standard for access control system. Offline access control systems do not need to be continuously connected to an active network to manage access rights. The "offline locks" are connected to the management system via the transponders. Access rights are written to and read from the card when the transponder is presented. Access rights are connected to the network via the transponders and updated when they come into contact with an active node.

Exactly what the data looks like and how it is structured varies from manufacturer to manufacturer. The OSS-SO is an effort of the OSS to offer a standard for "offline locks". The information is written to and read from the card according to the OSS Offline Standard.

Elatec supports a common interface that allows customers to flexibly implement OSS systems via OSDP.

2 Message Structure

The OSS commands shall be embedded within the data portion of a properly formatted OSDP packet, which is described in Section 2.9 of the OSDP Specification (Packet Format). This includes the correct length of packet, security block, and all other required parts of the OSDP frame.

OSDP Standard allows for definition of manufacturer-specific commands. The Host is able to send custom requests using the **osdp_MFG (code 0x80)** command. The Device will respond with a custom reply **osdp_MFGREP (code 0x90)**. If an error is detected, the **osdp_NAK (code 0x41)** can also be used.

The data portion of the **osdp_MFG (code 0x80)** command sent by Host shall have the format below.

Note that for simplicity the 3-Byte Vendor Code specified in OSDP Standard is intentionally not sent.

Byte	Name	Meaning	Value
0	Cmnd_ID	OSS command	any
1 .. 127	Data	(optional) Data related to command	any

Table 2.1: Structure of OSS command **osdp_MFG** requested by Host

The custom **osdp_MFGREP (code 0x90)** reply from the Device shall use Format below.

Byte	Name	Meaning	Value
0 .. 127	Data	(optional) Data related to response	any

Table 2.2: Structure of OSS response **osdp_MFGREP** sent by Device

Note that the Device could respond with an **osdp_BUSY (code 0x79)** if it is currently unavailable. The Host is expected to repeat the command with the Sequence Number **unchanged**.

3 OSS Commands

The OSS commands and the responses expected for them are listed in sections below. Note that any of the commands below could potentially receive a Negative Acknowledge reply **osdp_NAK (code 0x41)** if something goes wrong.

The process begins when the Reader finds a Transponder and communicates its UID to the Host via a simple **osdp_RAW (code 0x50)** command. At this point the Host can proceed with reading and/or writing data. Searching for Transponder will be paused for a number of seconds to allow the Host time to read/modify data.

It is assumed that the Application and File ID will not change from Transponder to Transponder. In order to speed up the Transponder processing time, the Reader will internally select the Application, the File and perform the necessary Authentication. This requires the Application and File IDs and Keys to be programmed into the Reader's App in advance.

3.1 OSS_GetFileSize

Command:	[Byte: <i>MFG</i>] [01] [Byte: <i>FileNo</i>]
Response:	[Byte: <i>MFGREP</i>] [Bool: <i>Result</i>] [(optional)UInt32: <i>FileSize</i>]
Example	
Command:	80 01 01
Response:	90 01 40 1D 00 00 (Result: true, file size: 7488 bytes)
Response:	90 00 (Result: false, File does not exist on Transponder)
	Note, osdp_NAK would not be used in this case - there were no OSDP errors involved.

3.2 OSS_ReadData

When it is known exactly what data is required - provide offset and length. **Maximum length is 120 Bytes.** Please see "Result" for the outcome of the read operation.

Command:	[Byte: <i>MFG</i>] [02] [Byte: <i>FileNo</i>] [UInt16: <i>Offset</i>] [UInt16: <i>Length</i>]
Response:	[Byte: <i>MFGREP</i>] [Byte: <i>Result</i>] (optional) [Byte Array(Var), 2 LB: <i>Data</i>]
Example	
Command:	80 02 01 07 00 05 00 (Offset: 7, Length: 5)
Response:	90 01 05 00 11 22 33 44 55 (Result: 0x01 (data read) , 5 Bytes, Data: 11 22 33 44 55)
Response:	90 02 03 00 11 22 33 (Result: 0x02 (not enough data in file) , 3 Bytes, Data: 00 11 22)
Response:	90 00 (Result: 0x00 (no data available)) Note, osdp_NAK would not be used in this case - there were no OSDP errors involved.

3.3 OSS_WriteData

When it is known exactly what data is written - provide offset and length. **Maximum length is 120 Bytes.** Please see "Result" for the outcome of the read operation. If the File size is smaller than number of Bytes to be written, then nothing will be written.

Command:	[Byte: <i>MFG</i>] [04] [Byte: <i>FileNo</i>] [UInt16: <i>Offset</i>] [Byte Array(Var), 2 LB: <i>Data</i>]
Response:	[Byte: <i>MFGREP</i>] [Byte: <i>Result</i>]
Example	
Command:	80 04 01 07 00 05 00 11 22 33 44 55 (FileNo: 1, Offset: 7, Length: 5, write 11 22 33 44 55)
Response:	90 01 (Result: 0x01 (data written))
Response:	90 00 (Result: 0x00 (no data written - either not enough space or file does not exist)) Note, osdp_NAK would not be used in this case - there were no OSDP errors involved.

3.4 OSS_Commit

Commit request shall have a simple true/false response. If operation failed for any reason, a simple **false** is returned.

Command:	[Byte: <i>MFG</i>] [06]
Response:	[Byte: <i>MFGREP</i>] [Bool: <i>Result</i>]
Example	
Command:	80 06
Response:	90 01 (Result: true, commit successful)
Response:	90 00 (Result: false, File does not exist on Transponder, or Authentication failed)

4 Disclaimer

ELATEC GmbH reserves the right to change any information or data in this document without prior notice. The distribution and the update of this document is not controlled. ELATEC GmbH declines all responsibility for the use of product with any other specifications but the ones mentioned above. Any additional requirement for a specific custom application has to be validated by the customer himself at his own responsibility. Where application information is given, it is only advisory and does not form part of the specification.

All referenced brands, product names, service names and trademarks mentioned in this document are the property of their respective owners.